COMPUTATIONAL PROBLEM SOLVING

Program Development Cycle

Introduction

- Computer can perform a variety of tasks like receiving data, processing it and producing useful results.

- Computer need to be instructed to perform even a simple task like addition of two numbers

- Computer work on a set of instructions called computer program

Introduction

- **Computer program:** specifies a way to carry out a task

- **Computer Programmer:** responsible for designing, writing, and modifying computer programs

  1. System Programmer: write programs, which provides interface and functionality to the hardware programs

  2. Application Programmer: writes programs to fulfil a specific task such as payroll system, inventory control

Developing a Program

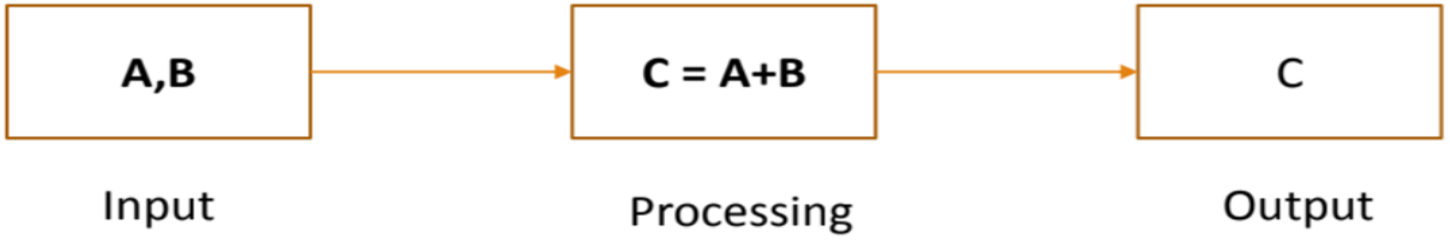
Program consists of a series of instructions that a computer processes to perform the required operation

Programmer should specifies:

1. The instruction to be performed

2. The order in which those instructions are to be performed

3. The data required to perform those instructions

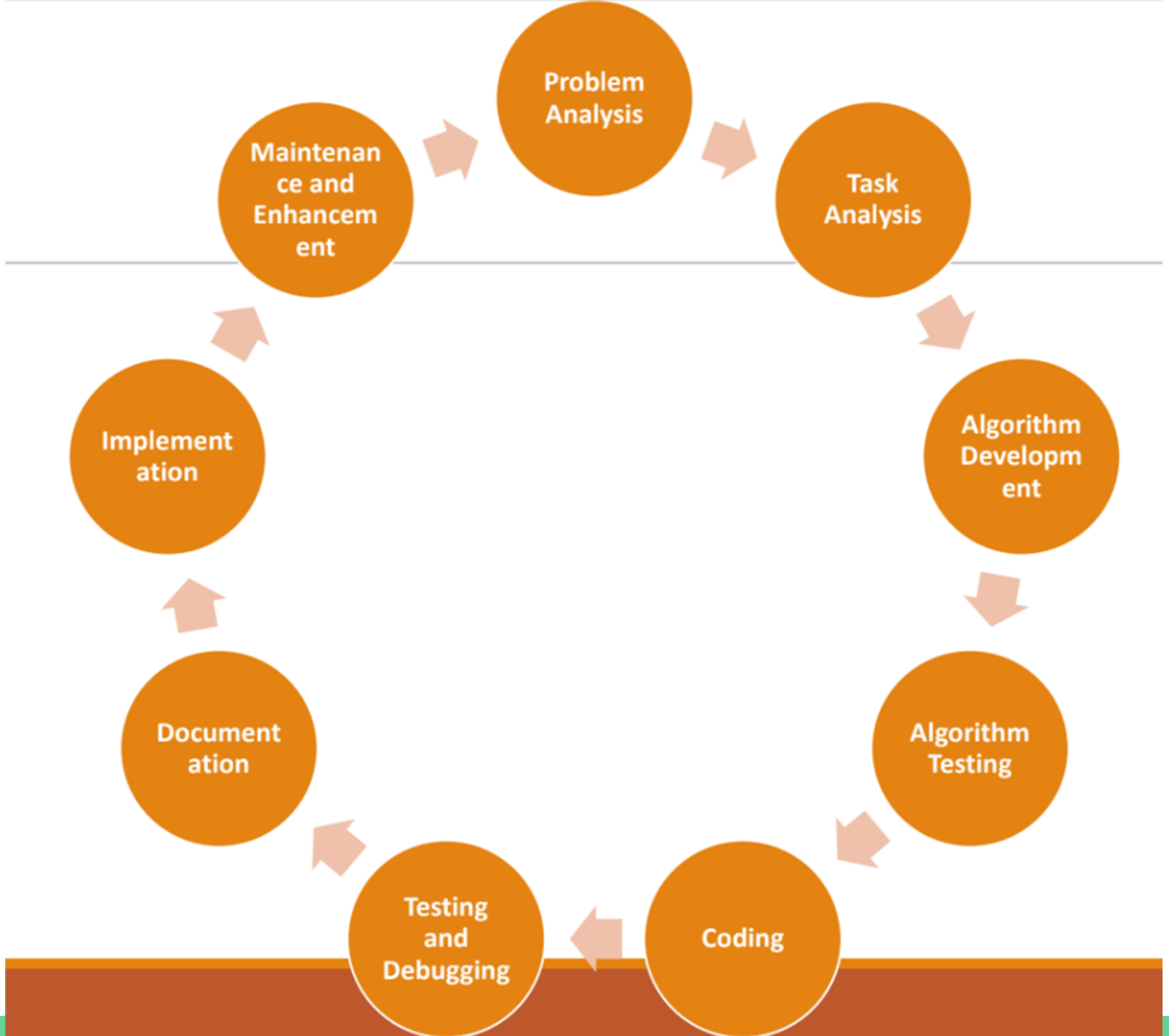


Program Development Cycle

- Before writing a program (coding), the programmer has to determine the problem that needs to be solved

- One common approach to problem solving is to use program development cycle

- While developing a program for a real-world problem, an organized plan should always be prepared. The program development concept focuses on dividing the process of building a program.


Problem Analysis
Task Analysis
Algorithm Development
Algorithm Testing
Coding
Testing and Debugging
Documentation
Implementation
Maintenance and Enhancement

Steps involved in Program Development Cycle

The program development concept defines seven phases/steps that each program goes through. For each phase, a certain set of steps is defined. The seven phases of program development are:

1. Problem Analysis
2. Task Analysis
3. Algorithm Development
4. Algorithm Testing
5. Coding
6. Testing and Debugging
7. Documentation
8. Implementation
9. Maitenance and Enhancement

Program Development Cycle

- **Problem Analysis:**
  - Problem is analyzed precisely and completely
  - Developer knows about the scope within the problem needs to developed

- **Task Analysis:**
  - Developer needs to develop various solutions to solve the given problem
  - From numerous solutions, optimum solution is chosen

Program Development Cycle

- **Algorithm Development:**
  - An algorithm (ex. flowchart) is developed to depict the basic logic of the selected solution
  - Algorithm depicts the solution in logical steps

Program Development Cycle

❖ **Algorithm Testing:**

◦ Before converting the algorithm into actual code, it should be checked for accuracy

◦ Test data need to be ‘walk through’ each step in the algorithm

  ◦ That verify that the instructions described in the algorithm actually perform the required functions or not

  ◦ Identify major logical errors, if any

  ◦ Ensure that algorithm work for both normal and unusual data

Program Development Cycle

- **Coding:**
  - Program takes place in the chosen programing language

- **Testing and Debugging:**
  - Find logical errors (semantic errors) or due to the incorrect use of programming language (syntax error)
  - Results obtained are compared with results calculated manually from the test data
  - Depend upon the complexity, several rounds of testing may be required

Program Development Cycle

- **Documentation:**
  - Once the program is free from the errors, it is the duty of programmer developer to ensure that program us supported by suitable documentation
  - Documentation enables the user to operate the program correctly
- **Implementation:**
  - The program is installed on the end user's machine
  - The implementation can be viewed as the final testing because only after using the program, the user can point out the drawbacks and report them to developers
- **Maintenance and Enhancement:**
  - Program should properly maintained by taking care of the changing requirements of its users and system

Pseudocode

Pseudocode

Flowcharts were the first design tool to be widely used, but unfortunately the do not very well reflect some of the concepts of structured programming.

Pseudocode, on the other hand , is a newer tool and has features that make it more reflective of the structured concepts. Unfortunately, the narrative presentation is not as easy to understand and follow.

Pseudocode

The first thing we do when designing a program is to decide on a name for the program.

Let’s say we want to write a program to calculate interest, a good name for the program would be **CalculatorInterest**.

Pseudocode

- So we start the program as:

```
PROGRAM CalculateInterest:
```

- And in general it's:

```
PROGRAM <ProgramName>:
```

Pseudocode

- Our program will finish with the following:

```
END.
```

- And in general it's the same:

```
END.
```

Pseudocode

- So the general structure of all programs is:

```
PROGRAM <ProgramName>:
<Do stuff>
END.
```

Pseudocode

- When we write a program, we assume that the computer executes the program starting at the beginning and working its way to the end.

- This is the basic assumption of all algorithm design.

- We call this **SEQUENCE.**

Pseudocode

```
PROGRAM MakeACupOfTea:
 Organise everything together;
 Plug in kettle;
 Put teabag in cup;
 Put water into kettle;
 Wait for kettle to boil;
 Add water to cup;
 Remove teabag with spoon/fork;
 Add milk and/or sugar;
 Serve;
END.
```

Pseudocode

- What ie we want to make a choice, for example, do we want to add sugar or not to the tea?

- We call this **SEQUENCE.**

Pseudocode

- So, we could state this as:

```
IF (sugar is required)
    THEN add sugar;
    ELSE don't add sugar;
ENDIF;
```

Pseudocode

- Adding a selection statement in the program:

```
PROGRAM MakeACupOfTea:
 Organise everything together;
 Plug in kettle;
 Put teabag in cup;
 Put water into kettle;
 Wait for kettle to boil;
 Add water to cup;
 Remove teabag with spoon/fork;
 Add milk;
 IF (sugar is required)
   THEN add sugar;
   ELSE do nothing;
 ENDIF;
 Serve;
END.
```

Pseudocode

- What if we need to tell computer to keep doing something until some condition occurs?

- Let’s say we wish to indicate that the you need to keep filling the kettle with water until it is full.

- We need a **loop**, or **ITERATION**.

Pseudocode

- So, we could state this as:

```
WHILE (Kettle is not full)
    DO keep filling kettle;
ENDWHILE;
```

Pseudocode

- Or to print out the numbers 1 to 5:

```
A = 1;
WHILE(A < 5)
    DO Print A;
        A = A + 1;
ENDWHILE;
```

- Pseudo code

Pseudo code can be broken down into five components.

- Variables:
- Assignment:
- Input/output:
- Selection:
- Repetition:

A variable has a name, a data type, and a value. There is a location in memory associated with each variable. A variable can be called anything or be given any name. It is considered good practice to use variable names that are relevant to the task at hand.

Assignment is the physical act of placing a value into a variable. Assignment can be shown using set = 5; set = num + set;

- Input / Output both deal with an outside source (can be a user or another program) receiving or giving information. An example would be assuming a fast food restaurant is a program. A driver (user) would submit their order for a burger and fries (input), they would then drive to the side window and pick up their ordered meal (output.) •
- Output – Write / display / print
- Input – Read / get / input Selection construct allows for a choice between performing an action and skipping it. It is our conditional statements.
- Selection statements are written as such:

if (conditional statement)
statement list
else
statement list

Repetition is a construct that allows instructions to be executed multiple times (IE repeated). In a repetition problem
– Count is initialized
– Tested
– incremented

Repetition problems are shown as:
while (condition statement)
statement list

Write pseudo code that reads two numbers and multiplies them together and print out their product.

Pseudo code:

PROGRAM Product
Read num1 , num2
Set multi to num1*num2
Write multi

Write pseudo code that tells a user that the number they entered is not a 5 or a 6.

Pseudo Code:

```
PROGRAM Num
READ isfive
IF(isfive = 5)
WRITE "your number is 5"
ELSE IF (isfive = 6)
WRITE "your number is 6"
ELSE
WRITE "your number is not 5 or 6"
```

Write pseudo code to print all multiples of 5 between 1 and 100 (including both 1 and 100).

Pseudo Code:

```
PROGRAM XYZ
SET x to 1
WHILE(x < 20)
WRITE x
x = x*5
```

Write pseudo code that will count all the even numbers up to a user defined stopping point.
For example, say we want to see the first 5 even numbers starting from 0.
well, we know that evens numbers are 0, 2, 4, etc.
The first 5 even numbers are 0, 2, 4, 6, 8.
The first 8 even numbers are 0, 2, 4, 6, 8 ,10 ,12, 16

Pseudo Code:

```
PROGRAM Num
READ count
SET x to 0;
WHILE(x < count)
SET even to even + 2
x = x + 1
WRITE even
```

Write pseudo code that reads in three numbers and writes them all in sorted order

Pseudo Code:

```
PROGRAM Sort
Read num1, num2, num3
If (num1 < num2)
      If(num2 < num3)
        Write num1 , num2, num3
      Else
          If(num3 < num1)
             Write num3, num1, num2
          Else
             Write num1, num3, num2
else
       If(num1 < num3)
            Write num2 , num1, num3
       Else
            If(num3 < num2)
              Write num3, num2, num1
            Else
              Write num2, num3, num1
```

Flowchart

Some Terminologies

- Algorithm / Flowchart
  - A step-by-step procedure for solving a particular problem.
  - Independent of the programming language.

- Program
  - A translation of the algorithm/flowchart into a form that can be processed by a computer.
  - Typically written in a high-level language like C, C++, Java, etc.

Variables and Constants

- Most important concept for problem solving using computers
- All temporary results are stored in terms of variables
  - The value of a variable can be changed.
  - The value of a constant do not change.
- Where are they stored?
  - In main memory.

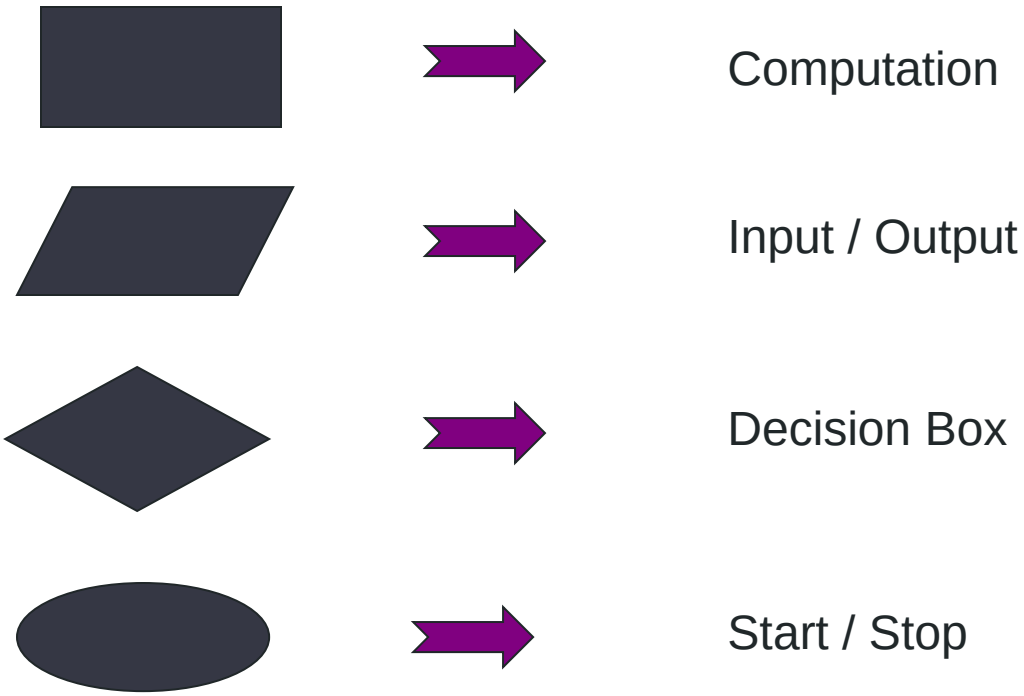
Variables and Constants…

- How does memory look like (logically)?
  - As a list of storage locations, each having a unique address.
  - Variables and constants are stored in these storage locations.
  - A variable is like a *bin*
    - The contents of the *bin* is the *value* of the variable
    - The variable name is used to refer to the value of the variable
    - A variable is mapped to a *location* of the memory, called its *address*

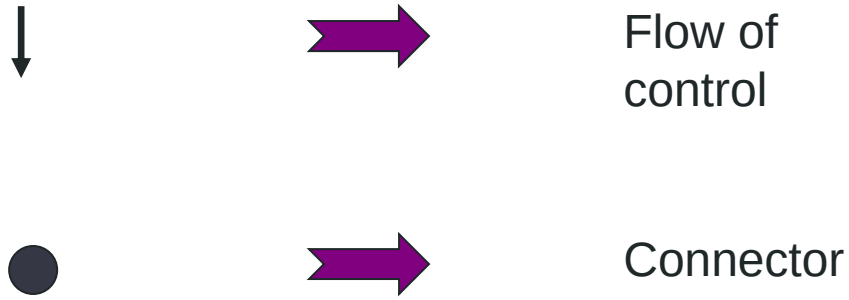
Data Types

- Three common data types used:
  - Integer :: can store only whole numbers
    - Examples: 25, -56, 1, 0
  - Floating-point :: can store numbers with fractional values.
    - Examples: 3.14159, 5.0, -12345.345
  - Character :: can store a character
    - Examples: ‘A’, ‘a’, ‘*’, ‘3’, ‘ ’, ‘+’

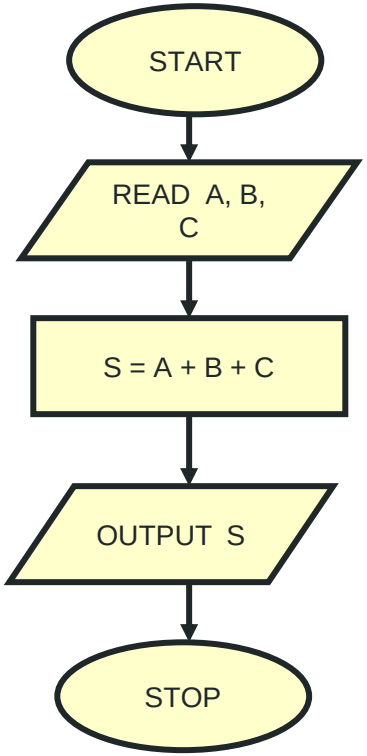
Flowchart: basic symbols



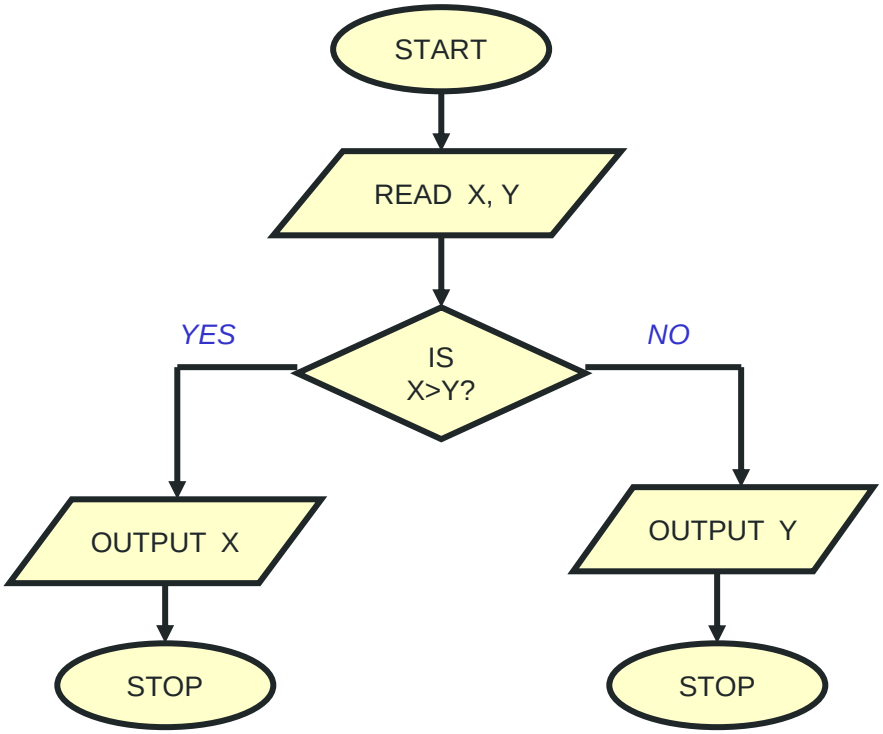
Flowchart: basic symbols...



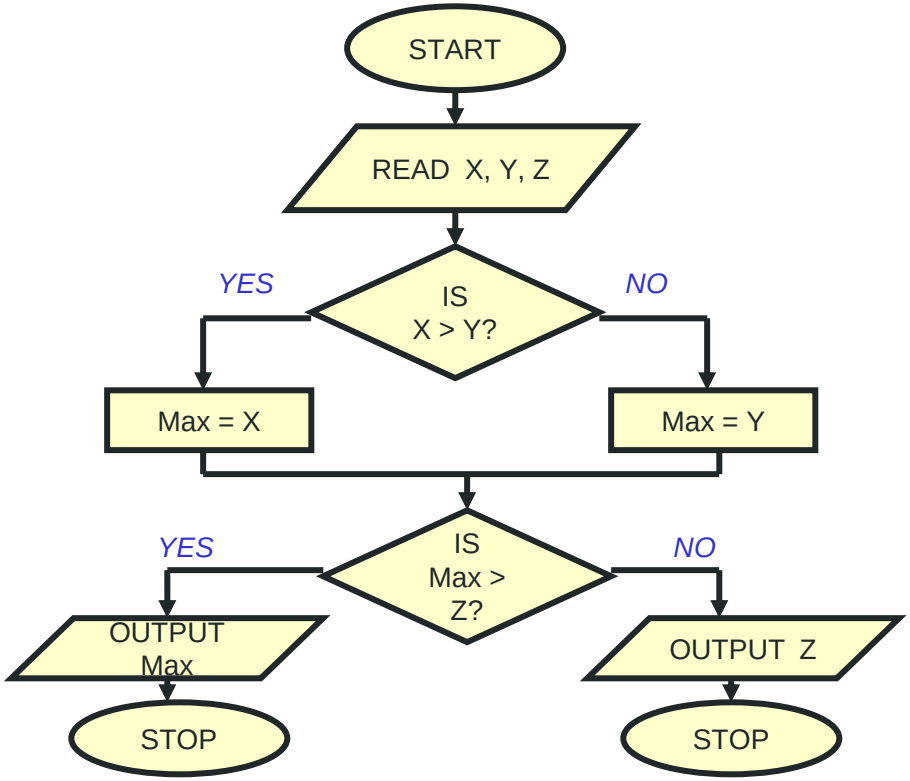
Example 1: *Adding three numbers*



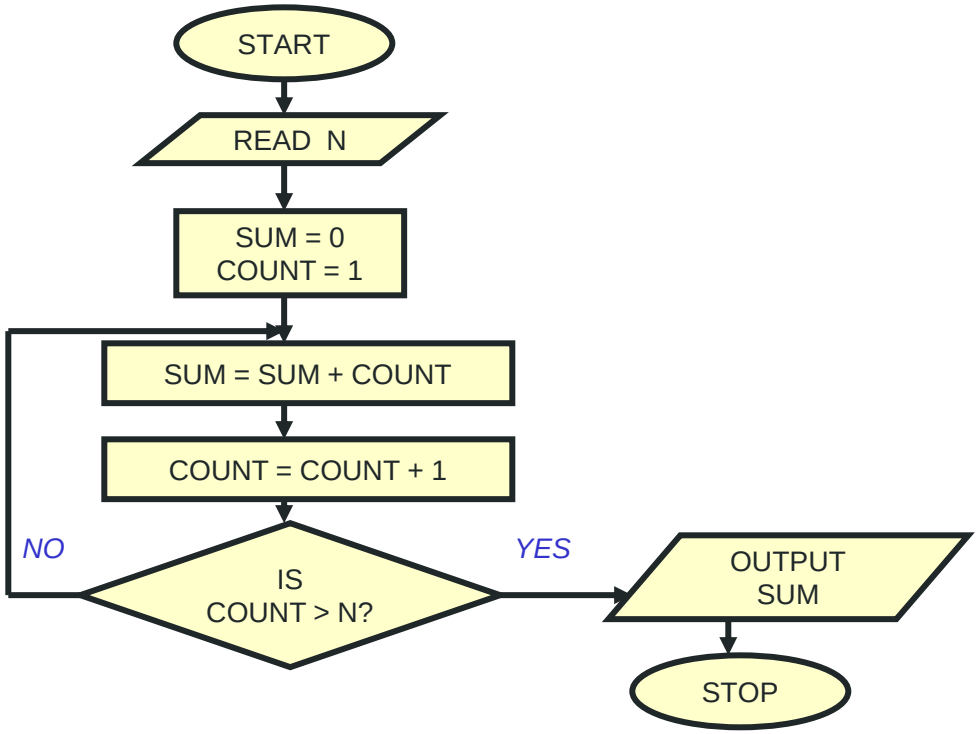
Example 2: *Larger of two numbers*



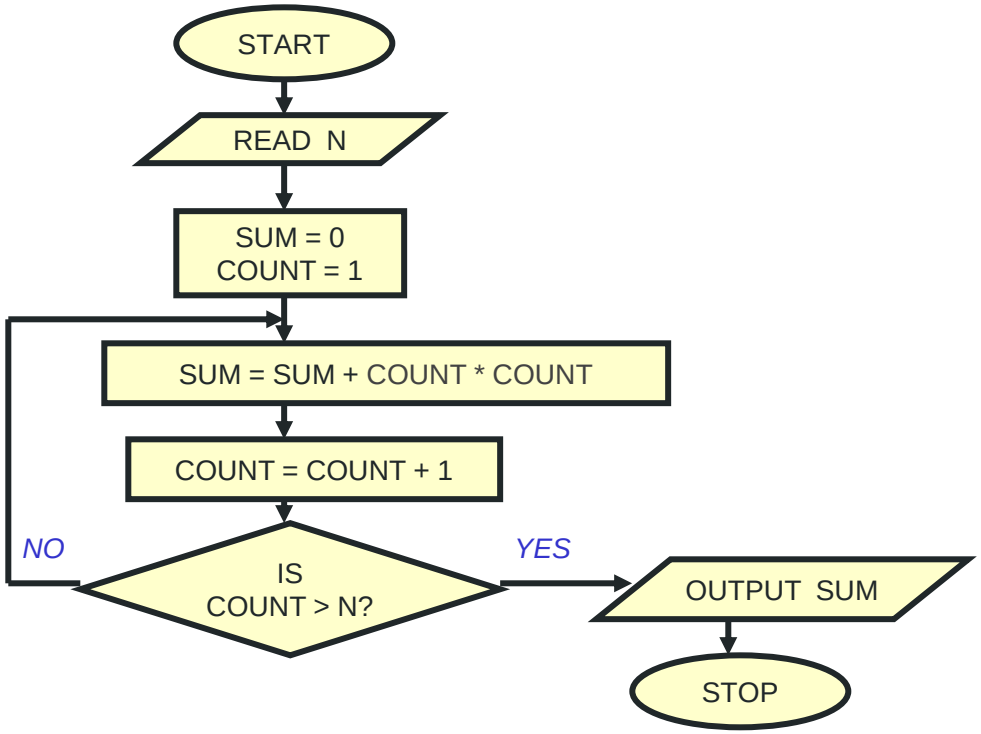
Example 3: *Largest of three numbers*



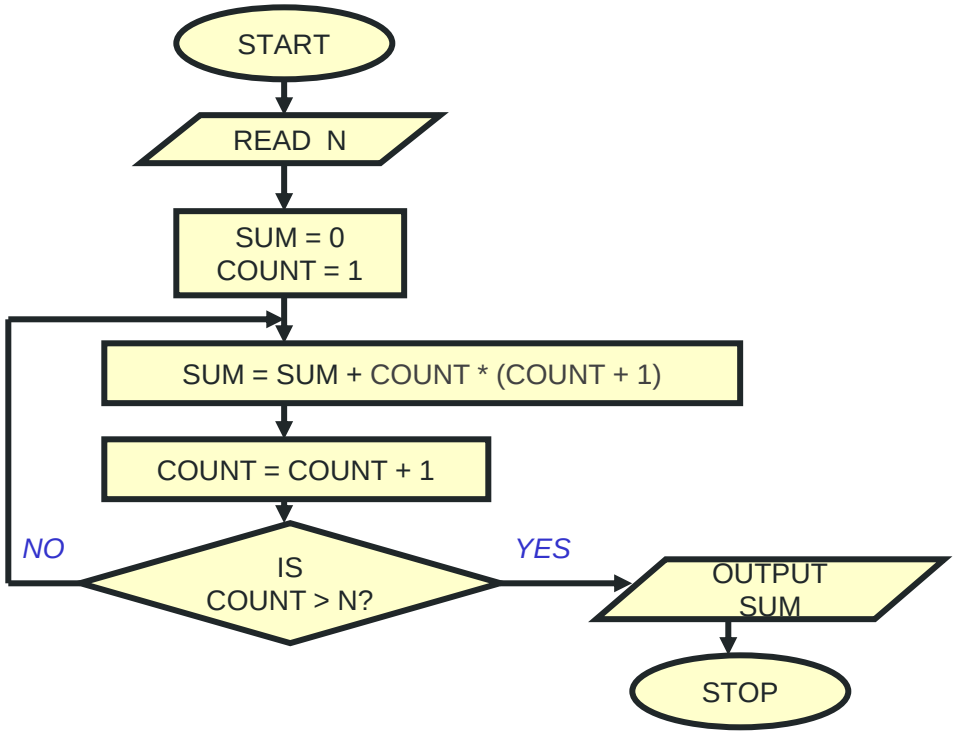
Example 4: *Sum of first N natural numbers*



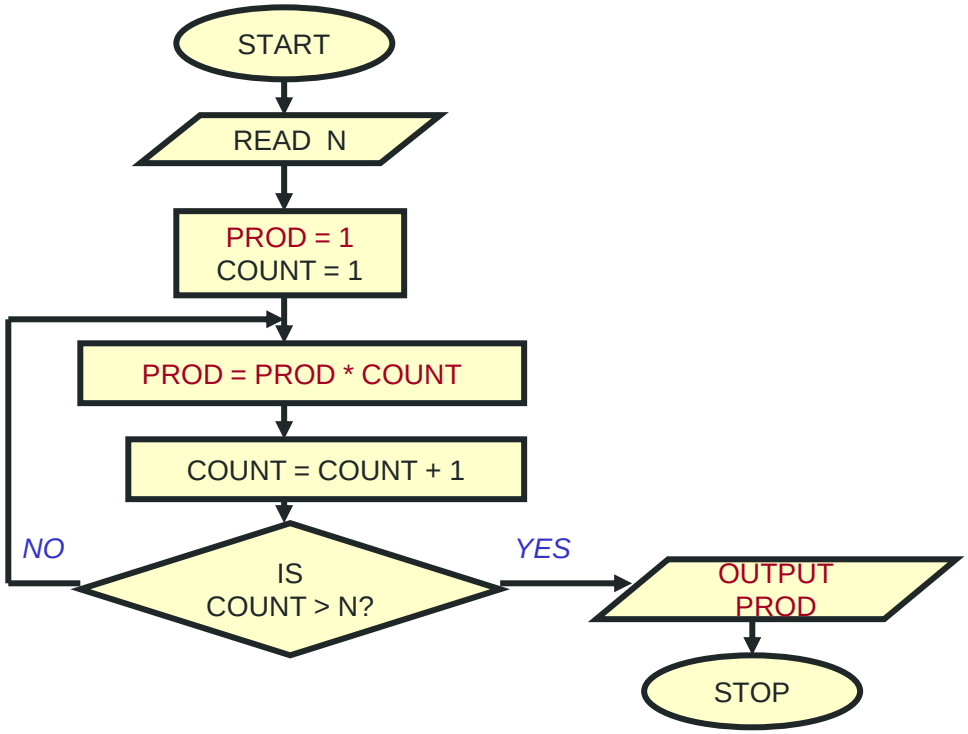
Example 5: $SUM = 1^2 + 2^2 + 3^2 + N^2$



Example 6: *SUM = 1.2 + 2.3 + 3.4 + to N terms*



Example 7: *Computing Factorial*



Example 8: *Roots of a quadratic equation*

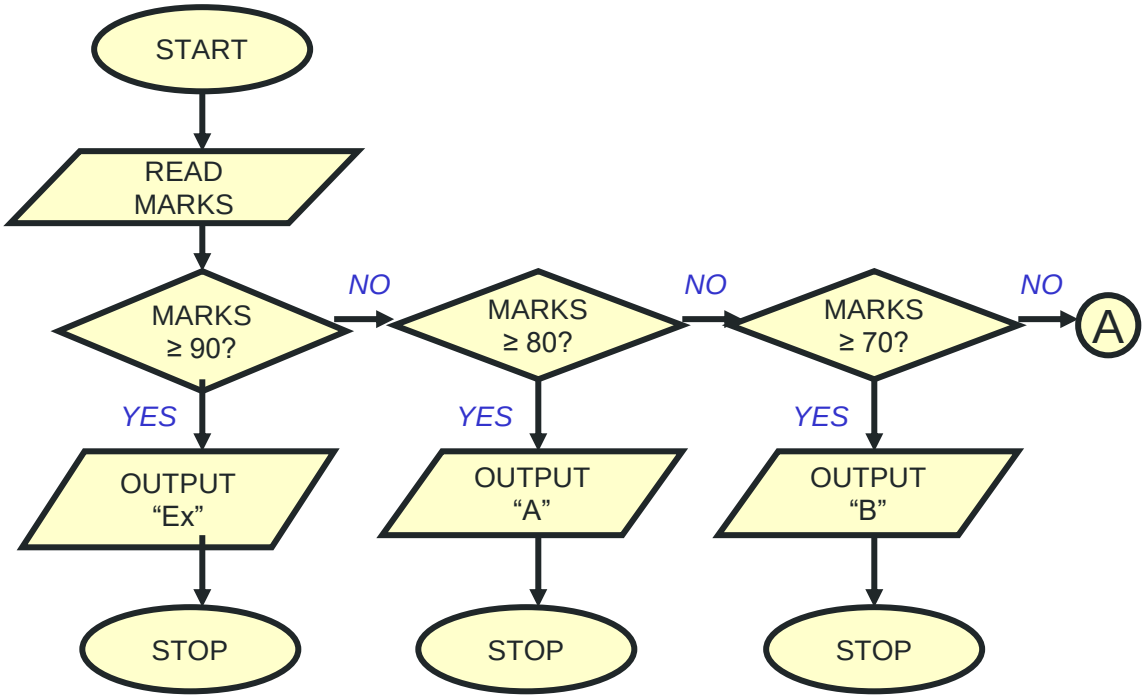
$$ax^2 + bx + c = 0$$

TRY YOURSELF

Example 9: *Grade computation*

- MARKS ≥ 90 Ex
- 89 ≥ MARKS ≥ 80 A
- 79 ≥ MARKS ≥ 70 B
- 69 ≥ MARKS ≥ 60 C
- 59 ≥ MARKS ≥ 50 D
- 49 ≥ MARKS ≥ 35 P
- 34 ≥ MARKS F

Example 9: *Grade computation…*



Representation Information as bits

- Representing Text
- Representing Numeric Value
- Representing Image
- Representing Sound

Representing Text

- Information in the form of text is normally represented by means of a code in which each of the text is assigned a unique bit pattern.
- The text is then represented as a long string of bits in which the successive symbols in the original text.

- ASCII – American Standard Code for Information Interchange
- A portion of ASCII is an eight bit per symbol format.

The following is a partial listing of ASCII code, in which each bit pattern has been extended with a 0 on its left to produce the eight-bit pattern commonly used today.

Symbol	ASCII	Symbol	ASCII	Symbol	ASCII
line feed	00001010	>	00111110	^	01011110
carriage return	00001101	?	00111111	_	01011111
space	00100000	@	01000000	a	01100001
!	00100001	A	01000001	b	01100010
"	00100010	B	01000010	c	01100011
#	00100011	C	01000011	d	01100100
$	00100100	D	01000100	e	01100101
%	00100101	E	01000101	f	01100110
&	00100110	F	01000110	g	01100111
'	00100111	G	01000111	h	01101000
(	00101000	H	01001000	i	01101001
)	00101001	I	01001001	j	01101010
*	00101010	J	01001010	k	01101011
+	00101011	K	01001011	l	01101100
,	00101100	L	01001100	m	01101101
-	00101101	M	01001101	n	01101110
.	00101110	N	01001110	o	01101111
/	00101111	O	01001111	p	01110000
0	00110000	P	01010000	q	01110001
1	00110001	Q	01010001	r	01110010
2	00110010	R	01010010	s	01110011
3	00110011	S	01010011	t	01110100
4	00110100	T	01010100	u	01110101
5	00110101	U	01010101	v	01110110
6	00110110	V	01010110	w	01110111
7	00110111	W	01010111	x	01111000
8	00111000	X	01011000	y	01111001
9	00111001	Y	01011001	z	01111010
:	00111010	Z	01011010	{	01111011
;	00111011	[	01011011	}	01111101
<	00111100	\	01011100		
=	00111101	]	01011101		

Representing Text

Example:
Message “Hello” in ASCII

01001000	01100101	01101100	01101100	01101111
H	e	l	l	o

Representing Text

- Unicode was developed through the cooperating of several of the leading manufactures of hardware and software
- It uses a unique pattern of 16 bits to represent each symbol.
- It consist of 65,536 different bit pattens.

Representing Numeric Values

- Binary Notation
- 1. Fraction in binary
- Two’s complement notation
- 1. Negative value
- 2. Length of the bit pattern
- Excess notation
- 1. Length of the bit pattern
- Floating point notation
- 1. Different field (sign, exponent, mantissa)
- 2. Radix point
- 3. Sign bit
- 4. Use 1 bytes, which is 8 bit length

Binary Representation

- Computer store and process all information in binary, a series bits, 1s and 0s.
  - Bit – Smallest element, either 0 and 1.
  - Byte – Grouping of 8 bits, 1101 0011
  - Kilobyte – 2^10 = 1024 bytes
  - Megabyte – 2^20 = 1048576 bytes
  - Gigabyte – 2^30 bytes
  - Terabyte – 2^40 bytes

Decimal Numbers

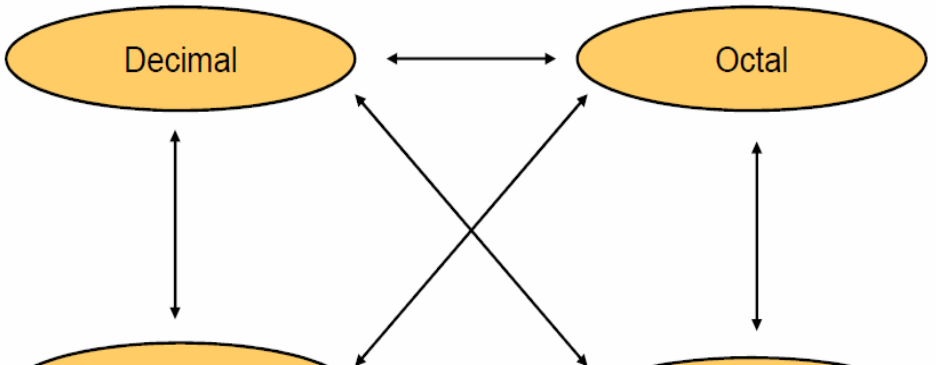
- How do we (usually) work with numbers?
  - Decimal system aka Base 10
  - Uses the 10 digits 0,1,2,3,4,5,6,7,8,9
  - $4357_{10} = 4000 + 300 + 50 + 7$
    $= 4 * 10^3 + 3 * 10^2 + 5 * 10^1 + 7 * 10^0$
  - Going from right to left each digits tells us the count for each power of 10 starting at 0

Other Number System

- Binary, Base 2
  - 2 digits, 0 and 1
  - $1010_2 = 1 * 2^3 + 0 * 2^2 + 1 * 2^1 + 0 * 2^0 = 8 + 2 = 10_{10}$
- Octal, Base 8
  - 8 digits, 0 thru 7
  - $35_8 = 3 * 8^1 + 5 * 8^0 = 24 + 5 = 29_{10}$
- Hexadecimal, Base 16
  - 16 digits, 0 thru 9,A,B,C,D,E,F
  - $12B_{16} = 1 * 16^2 + 2 * 16^1 + 11 * 16^0 = 256 + 32 + 11 = 299_{10}$

Conversion Among Bases

The possibilities:

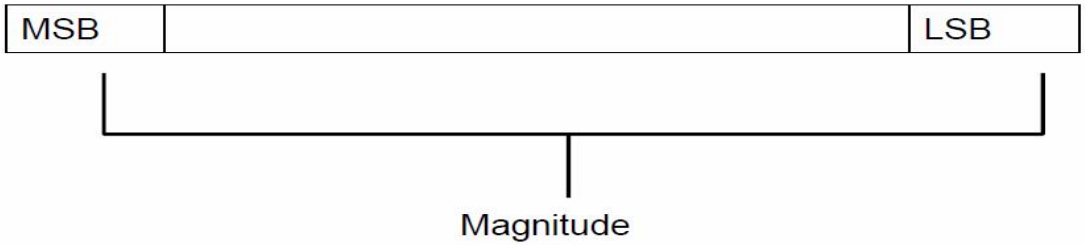


Binary

- How many numbers can be represented by a 4 digits decimal numbers?
- 1-9999 (10^5 - 1), 10^5 different number
- Binary 1-1111 (2^5 - 1) = 2^5 different number

Signed and Unsigned Numbers

UnSigned Number



Signed Number



If MSB = 0 → Positive number Magnitude

If MSB = 1 → Negative number



Two's Complement

- Two's complement is a binary representation usually used to represent positive and negative integers.
- Unlike the previous representation, every value has a single representation.
- First bit tell you whether given bit is positive – 0 or negative - 1.
- 7(2^4 - 1) is the largest possible number representable in 4 bit 2's complement (otherwise the first bit would be a 1).
- Positive number

0111 - 7

0010 - 2

- Negative number

1. 1 means negative number
2. (1. binary representation, 2. 1st complement, 3. 2nd complement)

   eg. 1111 = -0001 = -1

   1110 = -0010 = -2

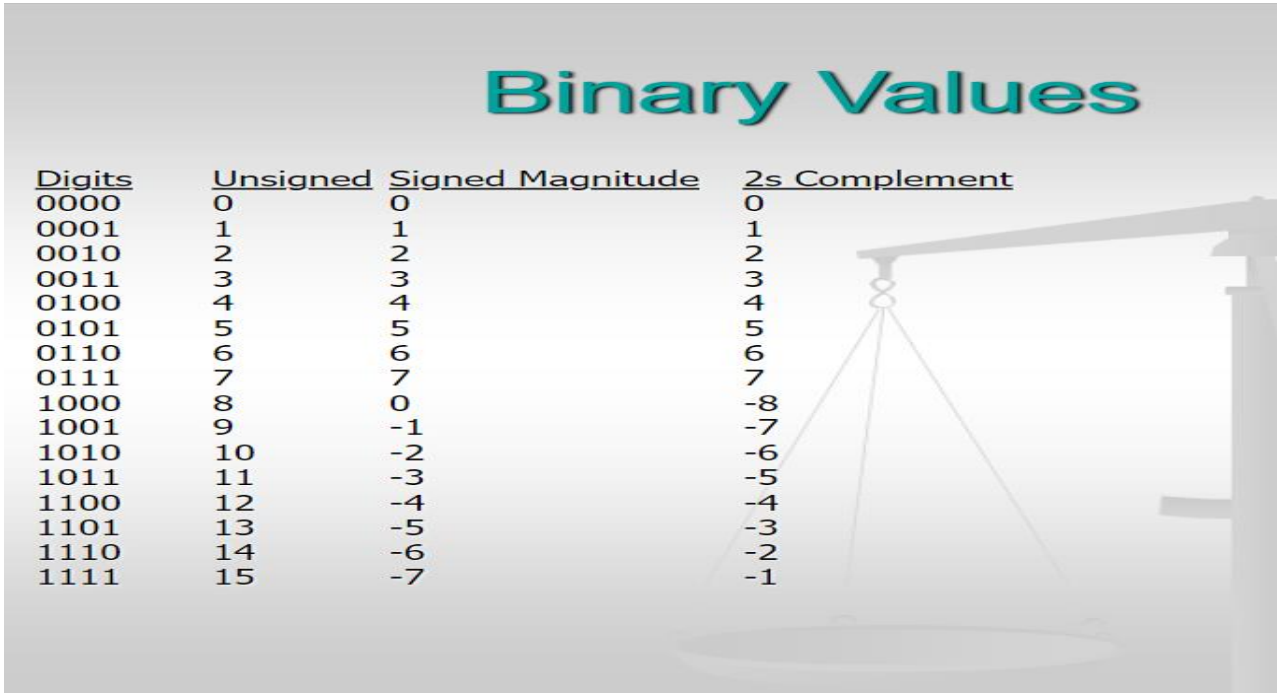
Two's Complement

- Advantages
  - As mentioned, single representation for each number
  - Single bit to check to know if number is positive or negative
  - For positive numbers, unsigned and 2's complement representation are the same
  - Simple arithmetic, addition and subtraction are done by (almost) the same method as normal addition and subtraction

Range

- What numbers can be represented given n bits (32 is typical for computers)
- Unsigned
  - 0 thru $(2^n - 1)$, 2^n different values
- Signed Magnitude
  - $-(2^{n-1} - 1)$ thru $(2^{n-1} - 1)$, $2^n - 1$ different values
- Two's Complement
  - -2^{n-1} thru $(2^{n-1} - 1)$, 2^n different values

Binary Values



Floating Point Number

- Floating point numbers (number that can have a decimal point) eg. 0.15, 0.4456 required more complex representation that we won't get into

ASCII Table

ASCII value	Character	Control character	ASCII value	Character	ASCII value	Character	ASCII value	Character
000	(null)	NUL	032	(space)	064	@	096	`
001	☺	SOH	033	!	065	A	097	a
002	☻	STX	034	"	066	B	098	b
003	♥	ETX	035	#	067	C	099	c
004	♦	EOT	036	$	068	D	100	d
005	♣	ENQ	037	%	069	E	101	e
006	♠	ACK	038	&	070	F	102	f
007	(beep)	BEL	039	'	071	G	103	g
008	◘	BS	040	(	072	H	104	h
009	(tab)	HT	041	)	073	I	105	i
010	(line feed)	LF	042	*	074	J	106	j
011	(home)	VT	043	+	075	K	107	k
012	(form feed)	FF	044	,	076	L	108	l
013	(carriage return)	CR	045	-	077	M	109	m
014	♫	SO	046	.	078	N	110	n
015	☼	SI	047	/	079	O	111	o
016	►	DLE	048	0	080	P	112	p
017	◄	DC1	049	1	081	Q	113	q
018	↕	DC2	050	2	082	R	114	r
019	‼	DC3	051	3	083	S	115	s
020	¶	DC4	052	4	084	T	116	t
021	§	NAK	053	5	085	U	117	u
022	▬	SYN	054	6	086	V	118	v
023	↨	ETB	055	7	087	W	119	w
024	↑	CAN	056	8	088	X	120	x
025	↓	EM	057	9	089	Y	121	y
026	→	SUB	058	:	090	Z	122	z
027	←	ESC	059	;	091	[	123	{
028	(cursor right)	FS	060	<	092	\	124	|
029	(cursor left)	GS	061	=	093	]	125	}
030	(cursor up)	RS	062	>	094	^	126	~
031	(cursor down)	US	063	?	095	_	127	⌂

Unicode

- A character set that has been design to store the character for all written language
- More bit required to stored the characters

Strings

- Simply store the ASCII values for characters in the string one after the other
- Using the ASCII character set
  - H = 72 = 01001000
  - i = 105 = 01101001
  - "Hi" = 01001000 01101001

Relationship among four kinds of coding system

decimal	binary	octal	hexadecimal
0	0000	000	0
1	0001	001	1
2	0010	002	2
3	0011	003	3
4	0100	004	4
5	0101	005	5
6	0110	006	6
7	0111	007	7
8	1000	010	8
9	1001	011	9
10	1010	012	A
11	1011	013	B
12	1100	014	C
13	1101	015	D
14	1110	016	E
15	1111	017	F
16	10000	020	10

Storing Integer

- Two's complement notation

The most popular means of representing integer values.

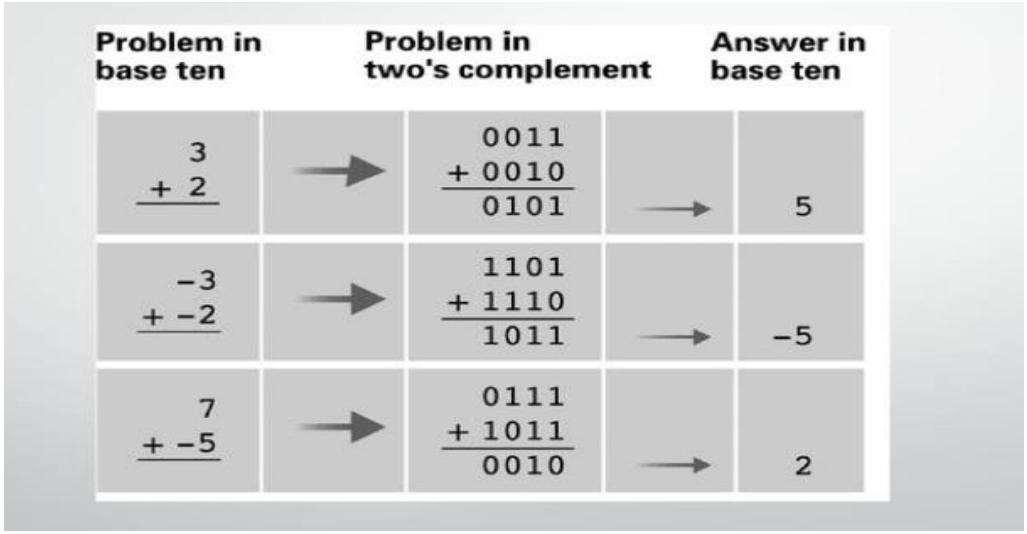
. Using patterns of length three

Bit pattern	Value represented
011	3
010	2
001	1
000	0
111	−1
110	−2
101	−3
100	−4

b. Using patterns of length four

Bit pattern	Value represented
0111	7
0110	6
0101	5
0100	4
0011	3
0010	2
0001	1
0000	0
1111	−1
1110	−2
1101	−3
1100	−4
1011	−5
1010	−6
1001	−7
1000	−8

Addition problems converted to two's complement notation



Problem in base ten		Problem in two's complement		Answer in base ten
3 + 2	→	0011 + 0010 0101	→	5
−3 + −2	→	1101 + 1110 1011	→	−5
7 + −5	→	0111 + 1011 0010	→	2

Storing Fractions

- **Floating point notation**

The storage of a value with a fractional part required that we store not only the pattern of 0s and 1s representing its binary representation but also the position of the radix point. A popular way of doing this is based on scientific notation and is called floating point notation.

Storing Fractions

- **Scientific notation**

N = 110.011 = 1.10011 * 2^10 = 11001.1 * 2^-10 = 0.110011 * 2^+11

Floating Point Notation consist of two fields:

1. The exponent field
2. The mantissa field

Single precision floating point: 32 bits, a precision of 7decimal digits.
Double precision floating point: 64 bits, a precision of 15 decimal digits.

The storage of single precision floating point

Exponent Field (1 Byte)		Mantissa Field (3 Byte)	
Sign of Exponent	**Exponent**	**Sign of Mantissa**	**Mantissa**
1 Bit	7 Bit	1 Bit	23 Bit
0	0000101	0	11010100000000000000000

$$26.5=11010.1=+0.110101\times2^{+5}$$

Excess Notation

Bit pattern	Value represented
111	3
110	2
101	1
100	0
011	−1
010	−2
001	−3
000	−4

Excess Eight Notation

Bit pattern	Value represented
1111	7
1110	6
1101	5
1100	4
1011	3
1010	2
1001	1
1000	0
0111	−1
0110	−2
0101	−3
0100	−4
0011	−5
0010	−6
0001	−7
0000	−8

- What is the decimal value of 10111100 in floating-point notation?
  - First bit is 1, then negative
  - Exponent is 011 and mantissa is 1100
  - Extract mantissa and place a radix point on its left side 0.1100
  - Extract exponent and interpret as excess four notation
  - 011 in excess four is -1 (make sure?)
  - -1 exponent means move radix point to the left by 1 bit
  - 0.1100 becomes 0.01100

- What is the decimal value of 01101011 in floating-point notation?
  - First bit is 0, then positive
  - Exponent is 110 and mantissa is 1011
  - Extract mantissa and place a radix point on its left side 0.1011
  - Extract exponent and interpret as excess four notation
  - 110 in excess four is +2 (make sure?)
  - +2 exponent means move radix point to the right by two bits (a negative exponent means move radix to left)
  - 0.1011 becomes 10.11

What Is Computer Science?

- **It is fundamentally about computational problem solving**
- **i.e., solving problems by the use of computation.**

What is computation?

- It is **any type of calculation that includes both arithmetical and non-arithmetical steps and follows a well-defined model**
- **E.g.: Algorithm.**

Types of Computational Problems

1. **Decision Problems**
2. **Search Problems**
3. **Counting Problems**
4. **Optimization Problems**

Computational thinking can be split into four parts,

1. **Decomposition**
2. **Pattern recognition**
3. **Abstraction**
4. **Algorithmic design**

- **Computational thinking allows us,**

1. **To take a complex problem**
2. **Understand what the problem is**
3. **Develop possible solutions**
4. **Present these solutions in a way that a computer, a human, or both, can understand.**

- **In order to solve a problem computationally, two things are needed,**

1. **A representation that captures all the relevant aspects of the problem**
2. **An algorithm that solves the problem by use of there presentation**

What Is an Algorithm?

- **It is** a **finite number of clearly described, unambiguous "doable" steps that can be systematically followed to produce a desired result for given input in a finite amount of time**
- **There are two main ways that algorithms can be represented,**

1. **Pseudocode**
2. **Flowcharts**

Pseudocode

- "Pseudo" -> **Imitation or False**
- "code" -> **Instructions written in a programming language**
- **Pseudocode -> will be a representation that almost looks like a code written in a programming language.**
- **It is also called Program Design Language (PDL)**
- **There are several formats which are used to write pseudo-codes and most of them take down the structures from languages such as C, Lisp, FORTRAN, etc.**
- **It allows you to include several control structures such as While, If-then-else, Repeat-until, for and case, which is present in many high-level languages**
- **It should be concise and the keyword should be in capital letter.**

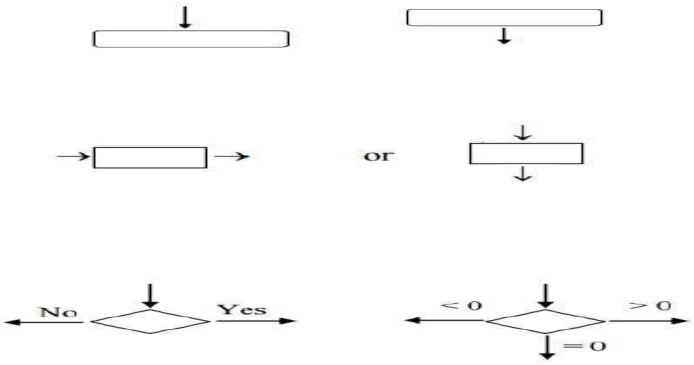
Flowchart

- **It is the graphical or pictorial representation of an algorithm**

Symbol	Name	Function
	Process	Indicates any type of internal operation inside the Processor or Memory
	input/output	Used for any Input / Output (I/O) operation. Indicates that the computer is to obtain data or output results
	Decision	Used to ask a question that can be answered in a binary format (Yes/No, True/False)
	Connector	Allows the flowchart to be drawn without intersecting lines or without a reverse flow.
	Predefined Process	Used to invoke a subroutine or an interrupt program.
	Terminal	Indicates the starting or ending of the program, process, or interrupt program.
	Flow Lines	Shows direction of flow.

General Rules for flowcharting

- **Only one flow line is used in conjunction with terminal symbol.**
- **Only one flow line should come out from a process symbol.**
- **Only one flow line should enter a decision symbol, but two or three flow lines, one for each possible answer, should leave the decision symbol.**



General Rules for flowcharting

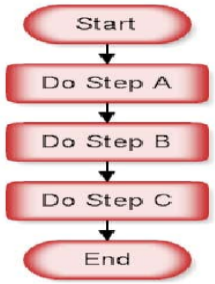
- Write within standard symbols briefly. As necessary, you can use the annotation symbol to describe dataor computational steps more clearly.
- Avoid the intersection of flow.
- If the flowchart becomes complex, it is better to use connector symbols to reduce the number of flowlines
- Ensure that the flowchart has a logical start and finish
- It is useful to test the validity of the flowchart bypassing through it with a simple test data.

Building Block of Algorithm

1. **Sequence structure**
2. **Decision Structure or Selection Structure**
3. **Repetition or Iteration Structure**

Sequence structure

- This is the most elementary structure
- In this, the steps in an algorithm are constructed in such a way that, no condition step is required
- It is the l



SEQUENCE

1.Compute the area and perimeter of a rectangle.

2.Accept three numbers from the user and find the average.

3.Evaluate the expression ((a*b) + (c / d))-f) by getting the values for each variable.

4.Calculate the distance walked by a person around a circular ground with a diameter of100m. (distance=πd)

SELECTION

1. Find the biggest among 2 numbers.

2. Find whether the given number is positive or negative.

3. Find the biggest among 3 numbers.

4. Find all roots of a quadratic equation

$ax^2+bx+c=0$. Read a number from the user. If the num is between 0 and 100, inclusive then display the message 'within range' else display the message 'out of range' otherwise.

ITERATION

1. **Display 1 ton natural numbers.**
2. **Find the factorial of a number entered by user.**
3. **Find the sum of integers between given two ranges.**
4. **Find the sum of numbers until the sum is less than 100.**
5. **Check whether a number entered by user is prime or not.**

Pseudocode:

START

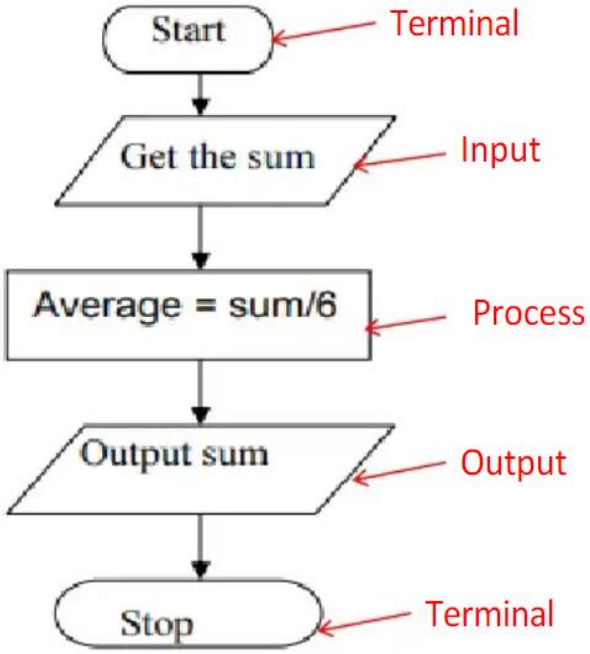
READ the sum

CALCULATE Average = sum / 6

OUTPUT the average

STOP

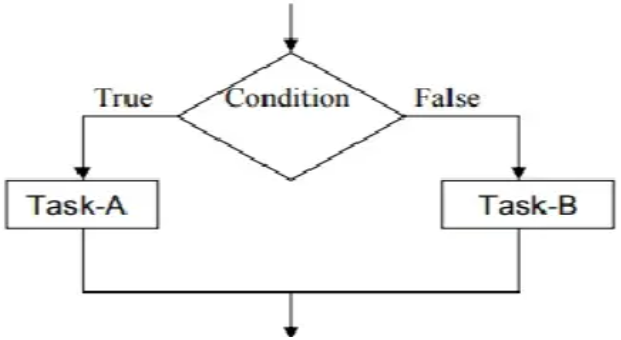
Flowchart:



Decision Structure or Selection Structure

- It is the case where in the algorithm, one has to **make a choice of two alternatives** by making decision depending on a given condition.
- Selection structures are **also called case selection structures whe there are two or more alternatives** to choose from

Flowchart

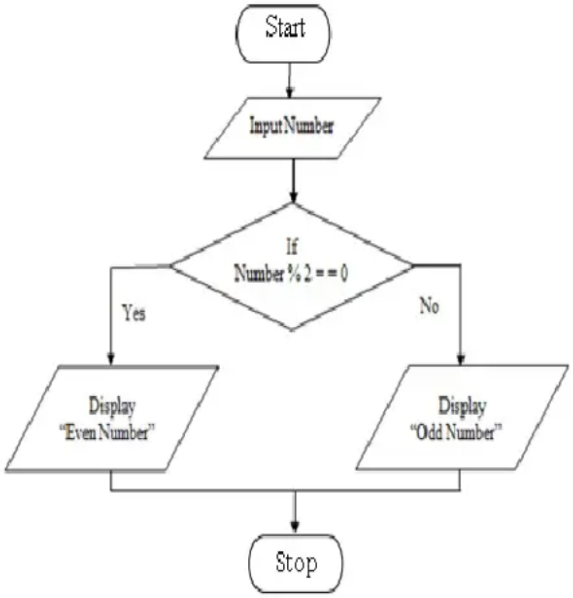


Pseudocode

```
IF condition is true THEN
        Do task A
ELSE
        Do Task-B
ENDIF
```

Problem: Determine and Output Whether Number N is Even or Odd

Flowchart



Pseudocode

START

READ number N

IF N modulo 2 is equal to 0 THEN

    Display “The number N is even”

ELSE

    Display “The number N is odd”

STOP

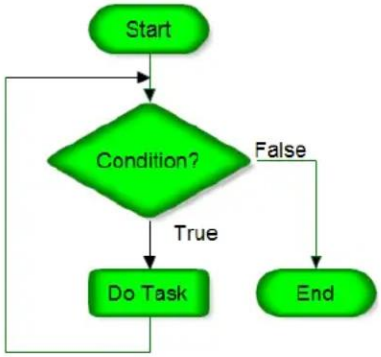
Repetition or Iteration Structure

- This structure causes the certain steps to be repeated
- •The Repetition structure can be implementedusing,

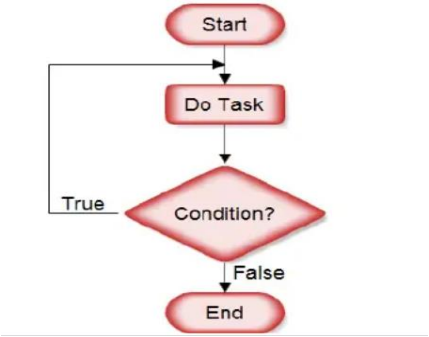
➢ The While Loop
➢ The For Loop
➢ Repeat Until Loop

- •Any program instruction that repeats somestatement or sequence of statements a number oftimes is called
- iteration or a loop
- •The commands used to create iterations or loopsare all based on
- logical tests
-

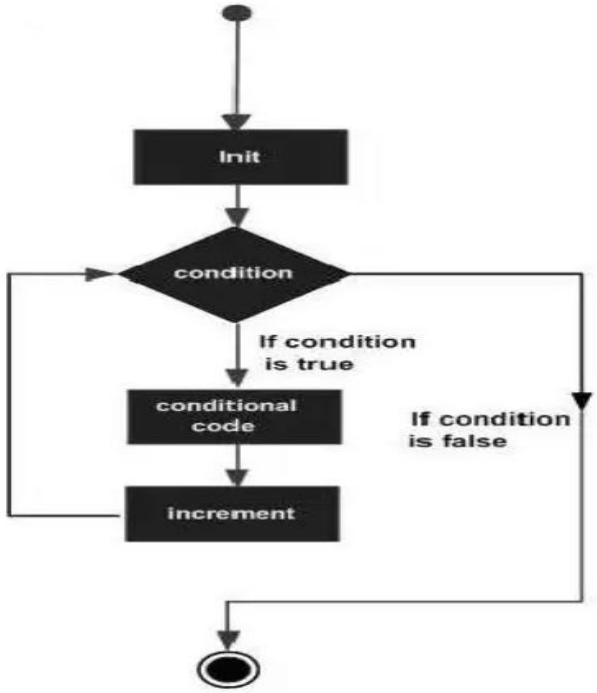
- **Pseudocode: (WHILE loop)**
- **WHILE (a condition is true)**
- **A statement or block ofstatements**
- **END WHILE**



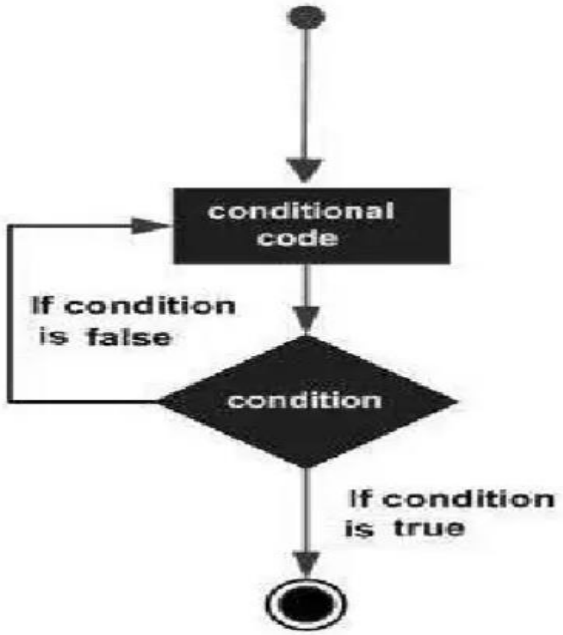
- **Pseudocode:(DOWHILE loop)**
- **DO**
- **A statement or block of statements**
- **WHILE (a condition is true)**
-



- **Pseudocode**
- **(FOR loop):**
- **FOR (Initialization, stopping condition, increment/decrement**
- **Statements**
- **END FOR**
-

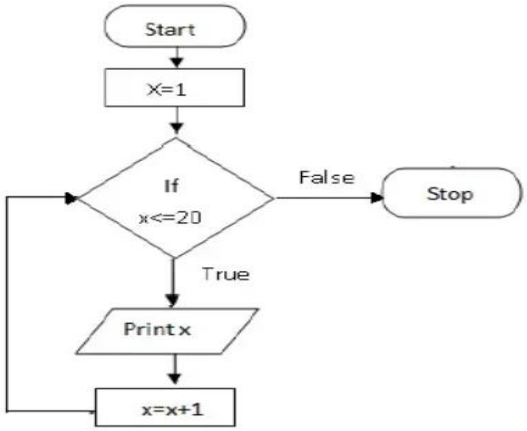


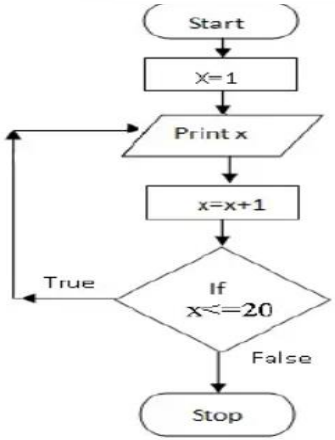
- **Pseudocode**
- **(REPEAT UNTIL)**:
- **REPEATA statement or block of statements**
- **UNTIL condition**
-

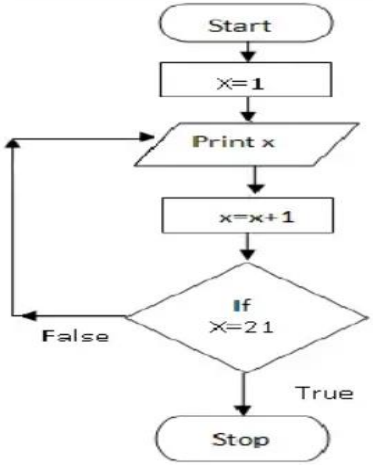


Problem:To print from 1 to 20

WHILE	FOR	DO WHILE	REPEAT UNTIL
SET x=1 WHILE (x<=20) PRINT x x=x+1 END WHILE	SET x=1 FOR(x, x<=20, x=x+1) PRINT x END FOR	SET x=1 DO PRINT x x=x+1 WHILE (x<=20)	SET x=1 REPEAT PRINT x x=x+1 UNTIL (x=21)







Convert Temperature from Fahrenheit (°F) to Celsius (°C)

Pseudocode:

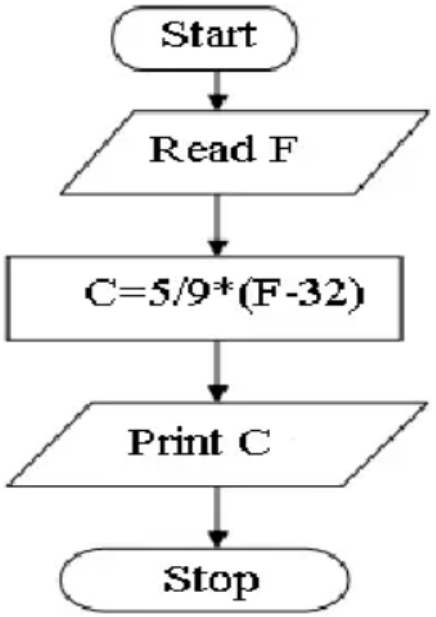
START

READ temperature in Fahrenheit (F)

CALCULATE Celcius (C) temperature
with formula C=5/9*(F-32),

PRINT C

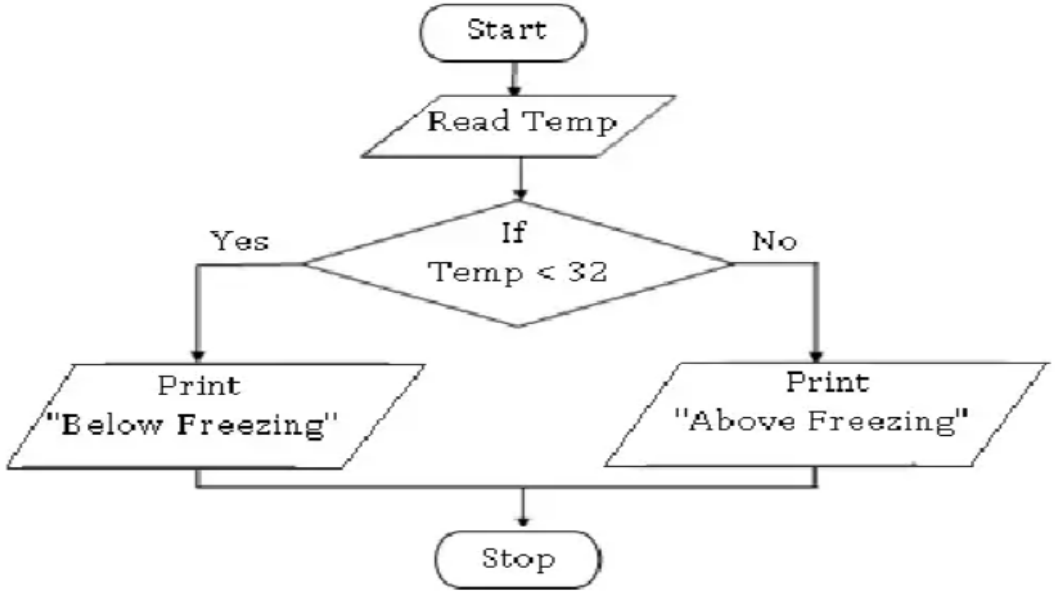
STOP



Determine Whether a Temperature is Below or Above the Freezing Point

Pseudocode:

```
START
INPUT temperature
IF (temperature <32) THEN
    PRINT "below freezing point"
ELSE
    PRINT "above freezing point"
STOP
```



Print sum of first 100 natural numbers

Pseudocode:

START
SET N=1, Sum=0
WHILE N<=100
    Sum=Sum+N
    N=N+1
END WHILE
PRINT Sum
STOP

